CSP-203: Software Tools & Technologies Lab



Chapter – 2

Automation using Scripting, Make and Cmake

(Lab Session – 9)

Dr. Anand M. Baswade
Indian Institute of Technology, Bhilai
anand@iitbhilai.ac.in

Contents

- ❑ Automation using scripting
- ❑ Automation using make
- ❑ Automation using cmake

❑ Automated compilation and execution of C program file over different inputs:

- Example:

```
arindam@ARINDAM:~$ cat hello.c
#include<stdio.h>
#include<stdlib.h>

void main(int argc, char* argv[]){
        if(argc<2)
        {
                printf("Input not provided!!!");
                exit(1);
        }
        int a = atoi(argv[1]);
        printf("Your entered number is: %d\n",a);
}
arindam@ARINDAM:~$ ./demo.sh
Executing for loop 0:
Your entered number is: 12
Executing for loop 1:
Your entered number is: 13
Executing for loop 2:
Your entered number is: 41
Executing for loop 3:
Your entered number is: 20
Executing for loop 4:
Your entered number is: 55
arindam@ARINDAM:~$
```

```
#! /bin/bash

array=(12 13 41 20 55)
gcc hello.c -o hello
for i in ${!array[@]}
do
        echo "Executing for loop $i:"
        ./hello ${array[$i]}
done
```

Manual compilation without any automation tool:

- ➔ compiler (gcc/g++) .c and .h files -o final_executable
- ➔ gcc -I. : Tells to include header library from the current working directory.
- ➔ If you don’t want to use -I. , you have to use #include “lib_header” instead of #include<lib_header>

Step 1:

all_lib.h — ~/Automation_Lab/simple_automation

```
1 void greetings();
2 int reverse(int num);
3 int factorial_num(int num);
```

Step 2:

greetings.c — ~/Automation_Lab/simple_automation

```
1 #include<stdio.h>
2 void greetings(){
3         printf("Hello IIT Bhilai...!!!");
4 }
```

Step 3:

reverse.c — ~/Automation_Lab/simple_automation

```
1 #include<all_lib.h>
2 int reverse(int num){
3         int sum=0;
4         while(num>0){
5                 int rev = num % 10;
6                 num /= 10;
7                 sum = sum*10 + rev;
8         }
9         return sum;
10 }
```

Step 4:

fact.c — ~/Automation_Lab/simple_automation

```
1 #include<all_lib.h>
2 int factorial_num(int num){
3         if(num==0){
4                 return 1;
5         }else{
6                 return num*factorial_num(num-1);
7         }
8 }
```

Step 5:

main.c — ~/Automation_Lab/simple_automation

```
1 #include<stdio.h>
2 #include<all_lib.h>
3
4 void main(){
5         greetings();
6         printf("\nReverse of 12345:
  %d\n",reverse(12345));
7         printf("Factorial of 5: %d\n",
  factorial_num(5));
8 }
```

Manual Compilation:

```
arindam@NAMS-LAB-NTN-401-System-1:~/Automation_Lab/simple_automatic
$ gcc -I. reverse.c fact.c greetings.c all_lib.h main.c -o main
arindam@NAMS-LAB-NTN-401-System-1:~/Automation_Lab/simple_automatic
$ ./main
Hello IIT Bhilai...!!!
Reverse of 12345: 54321
Factorial of 5: 120
arindam@NAMS-LAB-NTN-401-System-1:~/Automation_Lab/simple_automaaaa
arindam@NAMS-LAB-NTN-401-System-1:~/Automation_Lab/simple_automatic
$
```

What if we have 100000 .c files and main script and we need to compile and link them together?

- ➔ We will write gcc then 100000 file names, main script name the -o main ?
  - No …!!!
- ➔ We will use automation tool make
- ➔ Automates tasks like:
  - Compiling source code
  - Linking object files
  - Rebuilding only changed files

Common Makefile Macros

Variable	*Variable Description*
$@	The name of the executable file to be made
$?	The name of the changed dependencies
CC	Program to compile C programs, default is `gcc'
CXX	Program to compile C programs, default is `g++'
CFLAGS	Extra flags to be passed to C compiler
CXXFLAGS	Extra flags to be passed to C++ compiler
LDFLAGS	Extra flags to give to compilers when they are supposed to invoke the linker

❑ Compilation using make (without clean) :

Open

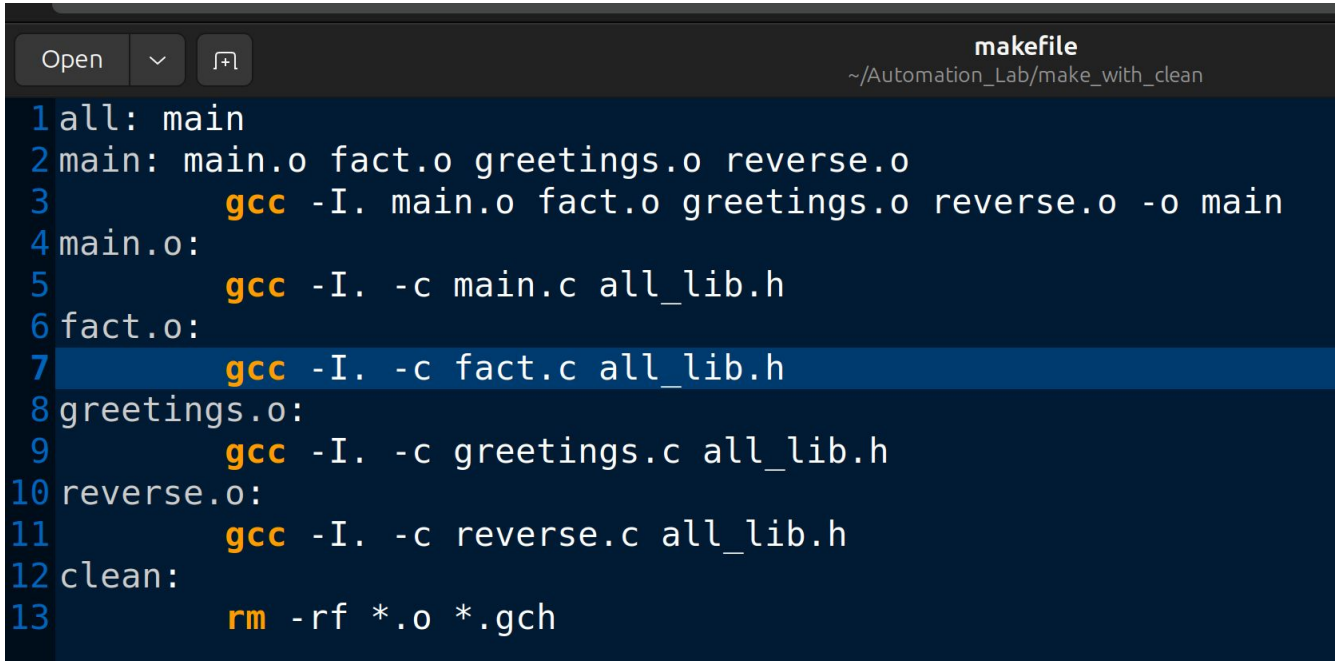
makefile
~/Automation_Lab/make_without_clean

```
1  all: main
2  main: main.o fact.o greetings.o reverse.o
3          gcc -I. main.o fact.o greetings.o reverse.o -o main
4  main.o:
5          gcc -I. -c main.c all_lib.h
6  fact.o:
7          gcc -I. -c fact.c all_lib.h
8  greetings.o:
9          gcc -I. -c greetings.c all_lib.h
10 reverse.o:
11         gcc -I. -c reverse.c all_lib.h
```

➔ Use the command to run Make: make

➔ For cleaning use: rm -rf *.o *.gch main

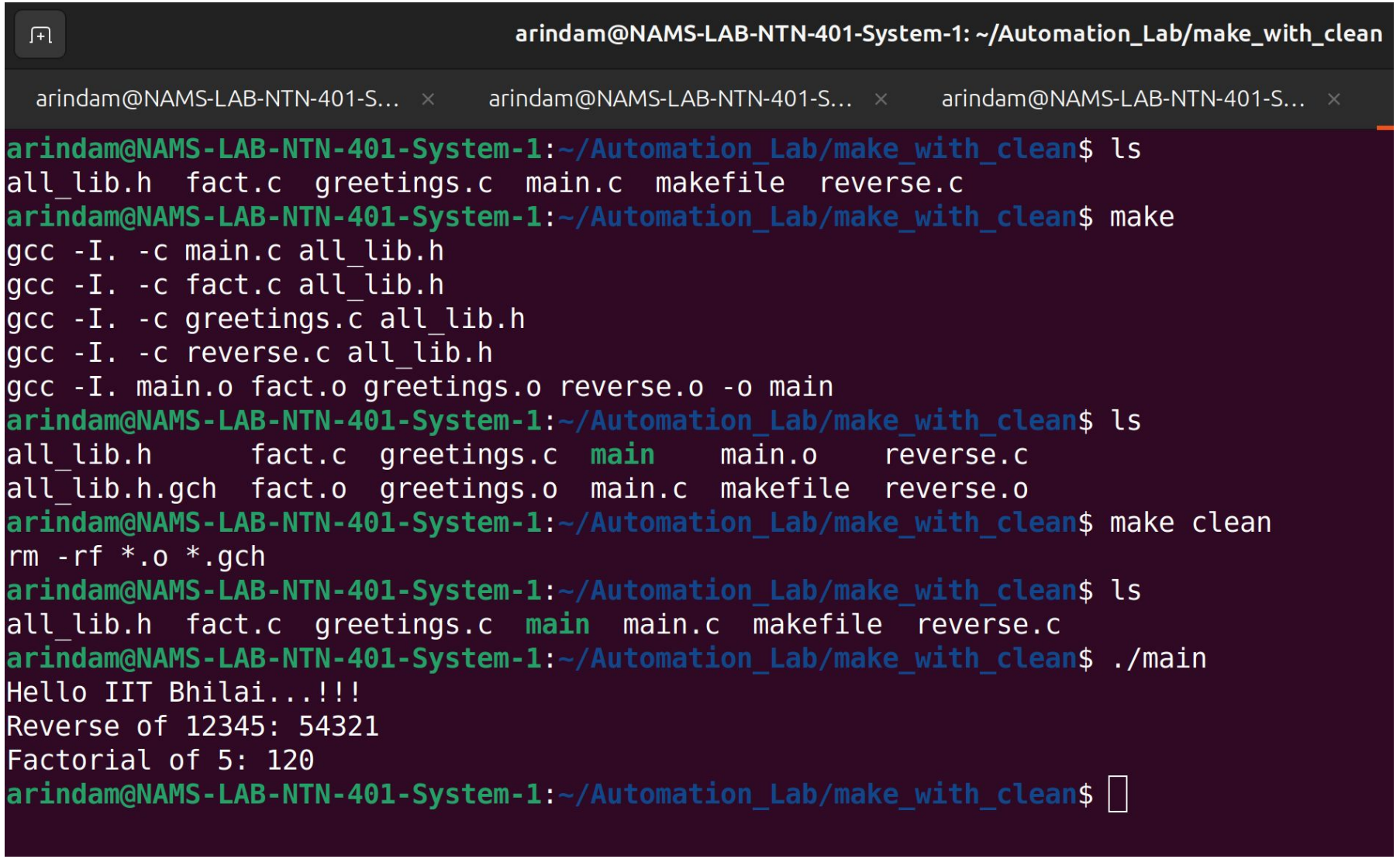
❑ Compilation using make (with clean) :



➔ Clears the generated intermediate object files if the following command is run:

make clean

❑ Output:


arindam@NAMS-LAB-NTN-401-System-1: ~/Automation_Lab/make_with_clean
arindam@NAMS-LAB-NTN-401-S...
arindam@NAMS-LAB-NTN-401-S...
arindam@NAMS-LAB-NTN-401-S...
arindam@NAMS-LAB-NTN-401-System-1:~/Automation_Lab/make_with_clean$ ls
all_lib.h fact.c greetings.c main.c makefile reverse.c
arindam@NAMS-LAB-NTN-401-System-1:~/Automation_Lab/make_with_clean$ make
gcc -I. -c main.c all_lib.h
gcc -I. -c fact.c all_lib.h
gcc -I. -c greetings.c all_lib.h
gcc -I. -c reverse.c all_lib.h
gcc -I. main.o fact.o greetings.o reverse.o -o main
arindam@NAMS-LAB-NTN-401-System-1:~/Automation_Lab/make_with_clean$ ls
all_lib.h fact.c greetings.c main main.o reverse.c
all_lib.h.gch fact.o greetings.o main.c makefile reverse.o
arindam@NAMS-LAB-NTN-401-System-1:~/Automation_Lab/make_with_clean$ make clean
rm -rf *.o *.gch
arindam@NAMS-LAB-NTN-401-System-1:~/Automation_Lab/make_with_clean$ ls
all_lib.h fact.c greetings.c main main.c makefile reverse.c
arindam@NAMS-LAB-NTN-401-System-1:~/Automation_Lab/make_with_clean$ ./main
Hello IIT Bhilai...!!!
Reverse of 12345: 54321
Factorial of 5: 120
arindam@NAMS-LAB-NTN-401-System-1:~/Automation_Lab/make_with_clean$

❑ Multiple makefiles in a directory :

Open — makefile1 — ~/Automation_Lab/multiple_makefiles

makefile1 | makefile2

```
1 all: main
2 main: main.o fact.o greetings.o reverse.o
3         gcc -I. main.o fact.o greetings.o reverse.o -o main
4 main.o:
5         gcc -I. -c main.c all_lib.h
6 fact.o:
7         gcc -I. -c fact.c all_lib.h
8 greetings.o:
9         gcc -I. -c greetings.c all_lib.h
10 reverse.o:
11        gcc -I. -c reverse.c all_lib.h
12 clean:
13        rm -rf *.o *.gch
```

makefile1

makefile1 | makefile2 | main.c

```
1 all: main
2 main: main1.o fact.o greetings.o reverse.o
3         gcc -I. main1.o fact.o greetings.o reverse.o -o main
4 main1.o:
5         gcc -I. -c main1.c all_lib.h
6 fact.o:
7         gcc -I. -c fact.c all_lib.h
8 greetings.o:
9         gcc -I. -c greetings.c all_lib.h
10 reverse.o:
11        gcc -I. -c reverse.c all_lib.h
12 clean:
13        rm -rf *.o *.gch
```

makefile2

Open — main.c — ~/Automation_Lab/multiple_makefiles

makefile1 | makefile2 | main.c

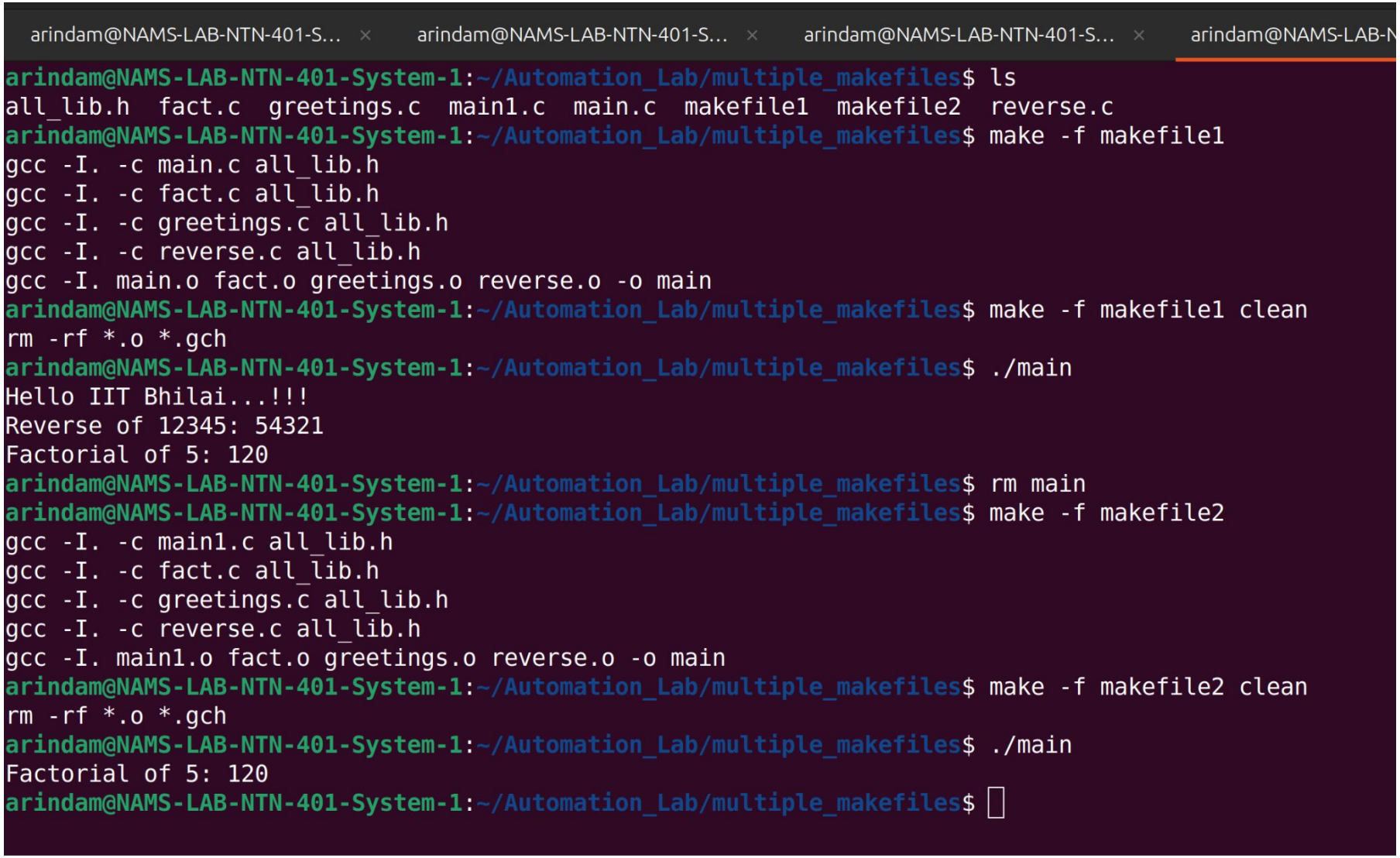
```
1 #include<stdio.h>
2 #include<all_lib.h>
3
4 void main(){
5         greetings();
6         printf("\nReverse of 12345: %d\n",reverse(12345));
7         printf("Factorial of 5: %d\n", factorial_num(5));
8 }
```

main.c

```
1 #include<stdio.h>
2 #include<all_lib.h>
3
4 void main(){
5         printf("Factorial of 5: %d\n", factorial_num(5));
6 }
```

main1.c

❑ Output:


arindam@NAMS-LAB-NTN-401-S...
arindam@NAMS-LAB-NTN-401-S...
arindam@NAMS-LAB-NTN-401-S...
arindam@NAMS-LAB-N
arindam@NAMS-LAB-NTN-401-System-1:~/Automation_Lab/multiple_makefiles$ ls
all_lib.h fact.c greetings.c main1.c main.c makefile1 makefile2 reverse.c
arindam@NAMS-LAB-NTN-401-System-1:~/Automation_Lab/multiple_makefiles$ make -f makefile1
gcc -I. -c main.c all_lib.h
gcc -I. -c fact.c all_lib.h
gcc -I. -c greetings.c all_lib.h
gcc -I. -c reverse.c all_lib.h
gcc -I. main.o fact.o greetings.o reverse.o -o main
arindam@NAMS-LAB-NTN-401-System-1:~/Automation_Lab/multiple_makefiles$ make -f makefile1 clean
rm -rf *.o *.gch
arindam@NAMS-LAB-NTN-401-System-1:~/Automation_Lab/multiple_makefiles$ ./main
Hello IIT Bhilai...!!!
Reverse of 12345: 54321
Factorial of 5: 120
arindam@NAMS-LAB-NTN-401-System-1:~/Automation_Lab/multiple_makefiles$ rm main
arindam@NAMS-LAB-NTN-401-System-1:~/Automation_Lab/multiple_makefiles$ make -f makefile2
gcc -I. -c main1.c all_lib.h
gcc -I. -c fact.c all_lib.h
gcc -I. -c greetings.c all_lib.h
gcc -I. -c reverse.c all_lib.h
gcc -I. main1.o fact.o greetings.o reverse.o -o main
arindam@NAMS-LAB-NTN-401-System-1:~/Automation_Lab/multiple_makefiles$ make -f makefile2 clean
rm -rf *.o *.gch
arindam@NAMS-LAB-NTN-401-System-1:~/Automation_Lab/multiple_makefiles$ ./main
Factorial of 5: 120
arindam@NAMS-LAB-NTN-401-System-1:~/Automation_Lab/multiple_makefiles$

Make using Macros:

Open ▾ makefile
~/Automation_Lab/make_using_macros

```
1  CC = gcc
2  TARGET = main
3  all: $(TARGET)
4  CFLAGS += -Wall
5  CFLAGS += -O2
6  CFLAGS += -I.
7  $(TARGET): main.o fact.o greetings.o reverse.o
8          $(CC) -I. main.o fact.o greetings.o reverse.o -o main
9  main.o:
10         $(CC) -I. -c $(CFLAGS) main.c all_lib.h
11 fact.o:
12         $(CC) -I. -c $(CFLAGS) fact.c all_lib.h
13 greetings.o:
14         $(CC) -I. -c $(CFLAGS) greetings.c all_lib.h
15 reverse.o:
16         $(CC) -I. -c $(CFLAGS) reverse.c all_lib.h
17 clean:
18         rm -rf *.o *.gch
```

arindam@NAMS-LAB-NTN-401-System-1: ~/Automation_Lab/make_using_macro

arindam@NAMS-LAB-NTN-401-S... × arindam@NAMS-LAB-NTN-401-S... × arindam@NAMS-LAB-NTN-401-S... ×

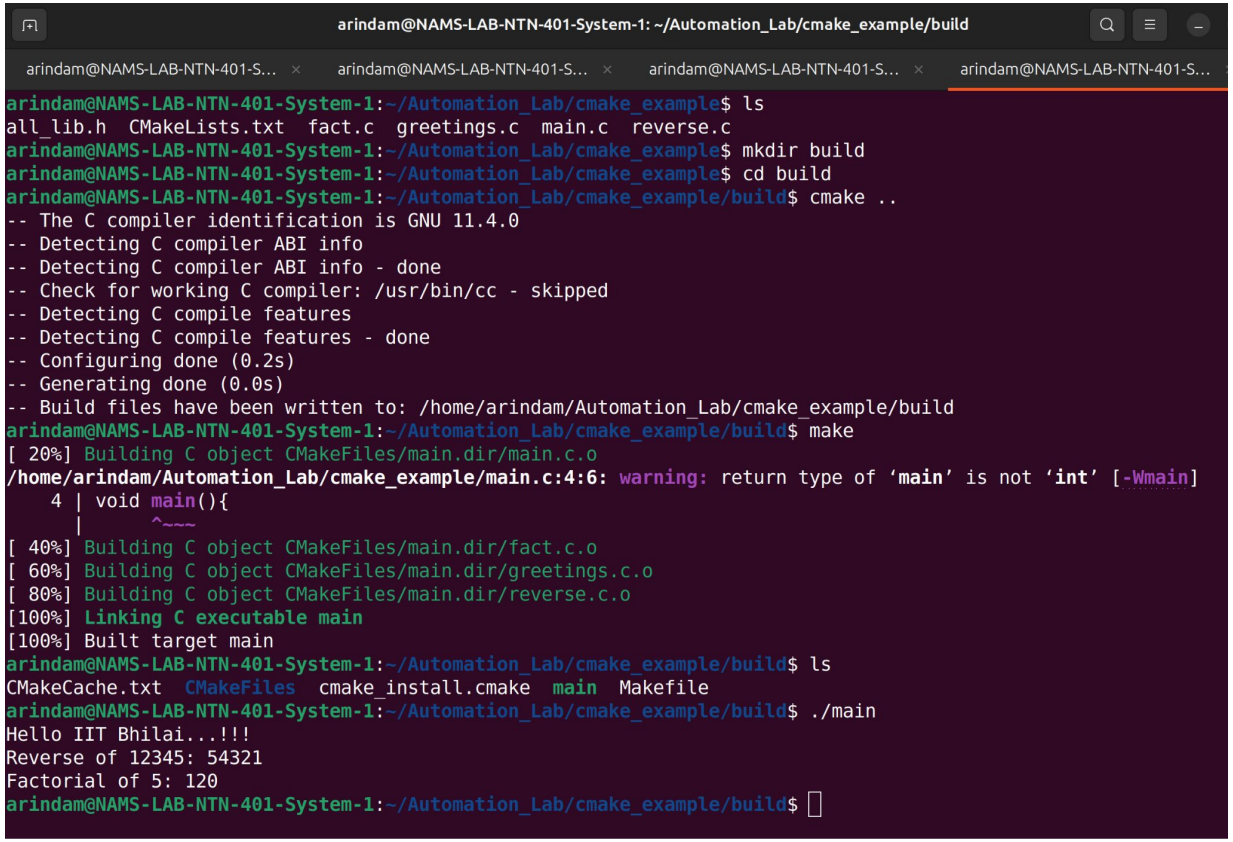
```
arindam@NAMS-LAB-NTN-401-System-1:~/Automation_Lab/make_using_macros$ ls
all_lib.h  fact.c  greetings.c  main.c  makefile  reverse.c
arindam@NAMS-LAB-NTN-401-System-1:~/Automation_Lab/make_using_macros$ gedit makefile
arindam@NAMS-LAB-NTN-401-System-1:~/Automation_Lab/make_using_macros$ make
gcc -I. -c -Wall -O2 -I. main.c all_lib.h
main.c:4:6: warning: return type of ‘main’ is not ‘int’ [-Wmain]
    4 | void main(){
      |      ^~~~
gcc -I. -c -Wall -O2 -I. fact.c all_lib.h
gcc -I. -c -Wall -O2 -I. greetings.c all_lib.h
gcc -I. -c -Wall -O2 -I. reverse.c all_lib.h
gcc -I. main.o fact.o greetings.o reverse.o -o main
arindam@NAMS-LAB-NTN-401-System-1:~/Automation_Lab/make_using_macros$ make clean
rm -rf *.o *.gch
arindam@NAMS-LAB-NTN-401-System-1:~/Automation_Lab/make_using_macros$ ls
all_lib.h  fact.c  greetings.c  main  main.c  makefile  reverse.c
arindam@NAMS-LAB-NTN-401-System-1:~/Automation_Lab/make_using_macros$ ./main
Hello IIT Bhilai...!!!
Reverse of 12345: 54321
Factorial of 5: 120
arindam@NAMS-LAB-NTN-401-System-1:~/Automation_Lab/make_using_macros$
```

.PHONY: all clean -> .PHONY tells make that the listed names are not actual files, but always to be treated as commands/targets.

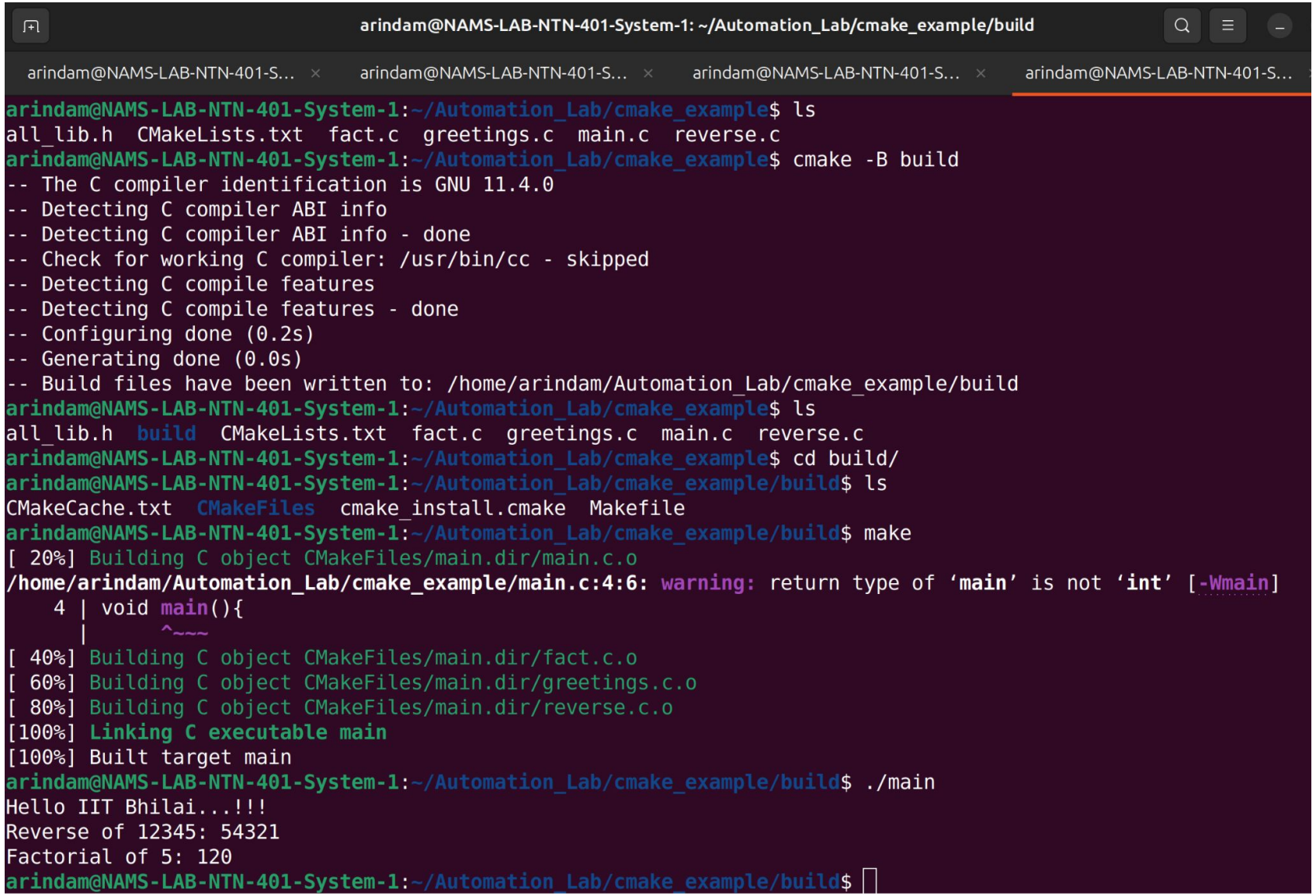
CMake Example:

Open ▾ CMakeLists.txt
~/Automation_Lab/cmake_example

```
1  cmake_minimum_required(VERSION 3.22)
2
3
4  project(MyProject C)
5
6
7  set(CMAKE_C_FLAGS "${CMAKE_C_FLAGS} -Wall -O2 -I.")
8
9  include_directories(${CMAKE_SOURCE_DIR})
10
11 add_executable(main
12     main.c
13     fact.c
14     greetings.c
15     reverse.c
16 )
```



CMake Example (Creating build directory automatically):




THANK
YOU